

Graph Attention in a Learned Branching Heuristic for SAT

Donato Meoli

Dipartimento di Informatica, Università di Pisa

Largo B. Pontecorvo 3, 56127 Pisa – Italy

donato.meoli@outlook.com

September 27, 2026

Abstract

We study the branching heuristic of a Conflict-Driven Clause Learning solver for the Boolean satisfiability problem when it is learned by reinforcement, as in Graph-Q-SAT, where a graph network reads the bipartite graph of the formula and is trained by Deep Q-Learning to reduce the number of decisions of the solver. We add to the core block of that network two edge-featured graph attention layers, which weight the neighbours of each node by learned coefficients before the global attribute is updated, and we compare the resulting model with the plain one at matched training set and configuration. On graph colouring the attention model takes fewer decisions than the plain one on all the families we test, and the difference tends to grow with the size of the instances, since the plain model degrades almost to the level of the reference solver on the largest ones. Trained on uniform random 3-SAT and tested on graph colouring, the attention model is again the better of the two, and it also appears to be the better one on some structured families that neither model has seen in training. Yet, on satisfiable uniform random 3-SAT the plain model is clearly superior, and on instances of this size neither model is competitive with the solver in running time, each decision costing a forward pass of the network. This evidence rests on two training seeds per variant on graph colouring, and on one or two elsewhere. Computational results are reported.

1 Introduction

We are concerned with the branching heuristic of a solver for the Boolean satisfiability problem (SAT), i.e., the problem of deciding whether a formula in conjunctive normal form

$$\varphi = \bigwedge_{j=1}^m \bigvee_{\ell \in C_j} \ell \quad (1)$$

over n Boolean variables admits an assignment that satisfies all the clauses C_j (each literal ℓ being a variable or its negation). We consider solvers that implement Conflict-Driven Clause Learning (CDCL): the search repeatedly (i) decides the value of an unassigned variable, (ii) propagates the assignments implied by the clauses, and (iii) upon a conflict learns a new clause and backjumps to the level that the clause allows [21, 20]. In practice the choice at point (i) is delegated to a heuristic, usually some variant of the Variable State Independent Decaying Sum (VSIDS) [25] (see, e.g., [18] for an alternative), which ranks the variables by activity counters that are increased at each conflict and periodically decayed. These counters usually carry little information at the start of the search. Furthermore, they are only a rough surrogate of what one would like to know (say, the variable whose assignment leads to the smallest search tree). The heuristic is therefore a natural place for a learned policy, and a convenient one, since a poor decision costs time but not correctness (the solver is sound and complete whatever the

policy does). We restrict ourselves to the choice of the variable and of its value, and we leave restarts, clause deletion and preprocessing as they are in MiniSat [6].

Learning for SAT has a large literature, which cannot be reviewed here; we refer to [11] for a survey and to [17] for a benchmark of graph networks on SAT problems, and only recall the works that bear on ours. NeuroSAT [28] predicts satisfiability from single-bit supervision, which shows that message passing [9] over the graph of a formula captures some of its structure. NeuroCore [27] predicts the variables that occur in unsatisfiable cores and uses the prediction to re-rank the VSIDS activities periodically, while NeuroBack [34] queries the network once, before the search, so that inference is paid offline. Other works learn heuristics for local search [35, 36]. Learning the branching rule by reinforcement goes back at least to [16], and the analogous problem for Mixed-Integer Programs has been approached with imitation learning and graph networks [14, 8] (cf. [3] for a survey of learning in combinatorial optimization). Graph-Q-SAT [15], on which we build, represents the state of the search as the bipartite graph of the formula, and learns the action-value function of the corresponding decision process with Deep Q-Learning (DQN) [23]. Since the network reads a graph, one set of weights applies to formulas of any size, and the heuristic can be evaluated on instances larger than (or different from) the training ones. This model has been extended with new graph representations and with “restricted” heuristics that query the network on part of the decisions only [29], and [24] studies how much of the reported gains depends on the experimental choices around the method. Graph attention [31, 4] appears in some of the works above but, to the best of our knowledge, not inside the core block of Graph-Q-SAT.

In this note we add to the core block of Graph-Q-SAT two graph attention layers that use the edge embeddings computed by the block, and we compare the resulting model (GAT-Q-SAT) with the plain one. We make the comparison at matched training set and configuration, on graph colouring (the family on which Graph-Q-SAT was originally trained, and larger instances of it), on uniform random 3-SAT and, with no retraining, on four structured families that neither model has seen. We remark that the attention layers are added to the block, the sum aggregation of Graph-Q-SAT being kept; hence, what we measure is the effect of the added layers (and of the parameters that come with them). We use two training seeds per variant on graph colouring and one or two on the other families, which is enough to see the larger differences but not to put an error bar on the smaller ones (i.e., those of a few tenths of the measure used below). As in [15] the results are measured in solver iterations, and the running time is reported separately. The note also documents a port of the original implementation to a current software stack (without which the experiments could not have been run), together with the test used to check that the port has not changed what the models compute.

The structure of the note is as follows. In section 2 we recall the decision process defined by the branching problem and the algorithm used to learn it, with the training configuration of all the runs. Section 3 describes the graph representation of the state and the network of Graph-Q-SAT, and section 4 the attention layers that GAT-Q-SAT adds to it; the port and the reproduction test are described in section 5. In section 6 we report the computational results, ordered by the distance between the training and the test instances (graph colouring at increasing size, a change of the training distribution, a change of domain), followed by uniform random 3-SAT and by the running time. Finally, in section 7 we draw some conclusions, and the appendix describes the instances and how the numbers of section 6 are regenerated. Instance families keep their SATLIB [12] names: `ufn-m` and `uufn-m` are satisfiable and unsatisfiable uniform random 3-SAT with n variables and m clauses, and `flatn-m` are the 3-colouring problems of graphs with n nodes and m edges, generated as in [5].

2 Branching as a decision process

Following [15], the solver is the environment of a Markov Decision Process. Its state is the formula (1) simplified by the current partial assignment (together with the clauses learned so far), and its actions are the pairs of an unassigned variable and a truth value. Transitions are computed by the solver itself (cf. [15, section 3]): a step is one decision followed by the propagation, the conflict analysis and the backjumping that it triggers, and therefore the transition is deterministic but known only by running it. A reward of $-p$ is given at each non-terminal step and 0 at termination (with $p = 0.1$ in all our runs); hence, maximizing the return means terminating in as few decisions as possible. Note that the formula is first simplified by the variable elimination of MiniSat, which is part of the environment.

The action-value function

$$Q^*(s, a) = \mathbb{E}[r + \gamma \max_{a'} Q^*(s', a')] \quad (2)$$

is approximated by a network Q_θ trained by DQN [23]. This minimizes the squared one-step temporal-difference error on minibatches drawn uniformly from a replay buffer, with the bootstrap target computed by a copy θ^- of the weights that is refreshed periodically, and with the transitions collected by an ε -greedy policy (neither the double estimator of [30] nor the prioritized replay of [26] is used); the learned heuristic is then $\pi(s) = \arg \max_a Q_\theta(s, a)$. Two features of the problem constrain the approximator: the number of actions changes along an episode (two per unassigned variable), and the state is defined up to a permutation of the variables and of the clauses. Hence, the network must score a variable number of actions and be invariant under those permutations, which rules out fixed-size encodings such as that of [33] and leads to the graph representation of section 3.

All the runs of this note use the configuration of [15], which we list here since it is not repeated elsewhere. Training uses Adam with a constant learning rate of $2 \cdot 10^{-5}$, the discount is $\gamma = 0.99$, the replay buffer holds 20000 transitions and the minibatches have 64 of them. One update is made every 4 transitions after the first 5000, and θ is copied into θ^- every 10 updates. Training stops after 50000 updates (about $2 \cdot 10^5$ transitions); ε is 1 for the first 5000 transitions and then decreases linearly to 0.01. During training an episode may take at most 500 decisions of the model, after which the solver completes the search with VSIDS in a single step. Also, the gradient norm is clipped at 0.1 for the runs trained on graph colouring and at 1 for those trained on uniform random 3-SAT (as in the respective configurations of [15]).

3 Graph-Q-SAT

The state is represented [15] as a bipartite graph whose nodes are the unassigned variables and the clauses (original and learned) that are not yet satisfied, each clause restricted to its literals that are not false. There is an edge between a variable and each clause in which it occurs (stored in both directions). Node features are 2-dimensional one-hot vectors that distinguish the two kinds of node, edge features encode in the same way the polarity of the literal, and the global attribute is a constant. On each variable node the network produces two Q -values on each variable node (one per truth value), and the agent takes the $\arg \max$ over all of them.

As in [15], the network is an encode-process-decode graph network [2] (cf. Figure 1). An encoder maps the edges, the nodes and the global attribute independently to embeddings of size 32. Then, the core block (whose weights are shared) is applied $M = 4$ times to the concatenation of the encoded graph and of its previous output. Each application updates first the edges, then the nodes, then the global attribute:

$$e'_k = \phi^e(e_k, v_{r_k}, v_{s_k}, u), \quad v'_i = \phi^v\left(\sum_{k: r_k=i} e'_k, v_i, u\right), \quad u' = \phi^u(\bar{e}', \bar{v}', u), \quad (3)$$

where r_k and s_k are the two endpoints of edge k , $\bar{e}'$ and $\bar{v}'$ are the means of the updated edge and node embeddings, and ϕ^e , ϕ^v and ϕ^u are perceptrons with one hidden layer of 64 units and outputs of size 64, each followed by a layer normalization. Finally, a decoder reads the Q -values off the variable nodes. Since the weights are shared across the nodes and across the M rounds, one set of them applies to a formula with any number of variables and clauses; with this configuration the network has 71144 parameters.

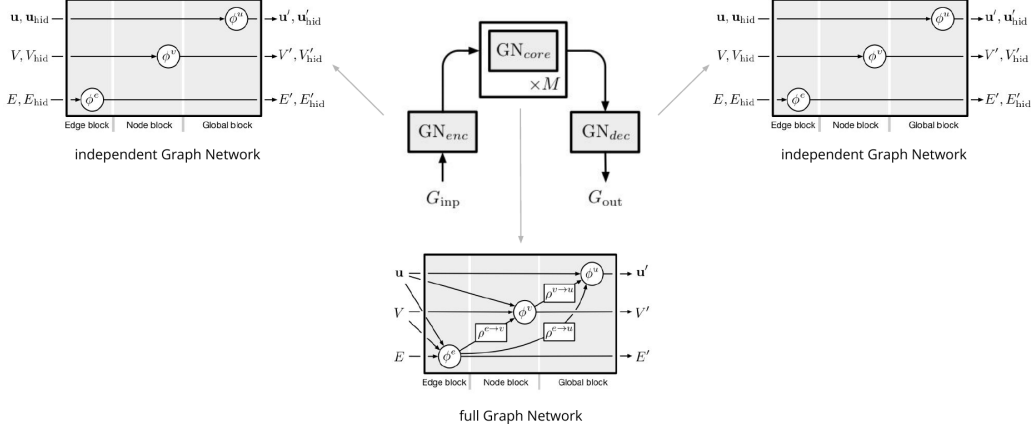


Figure 1: The encode–process–decode network of Graph-Q-SAT: an independent block encodes the edges, the nodes and the global attribute, a full graph-network block is applied M times with shared weights, and a second independent block decodes the Q -values. Blocks drawn after [2].

Note that the node update of (3) sums the messages of the edges incident to a node, and therefore it gives the same weight to all the clauses in which a variable occurs (and to all the variables of a clause). For a formula drawn uniformly at random at the phase transition [22] this seems a reasonable choice, the clauses being statistically alike. In a formula that encodes a structured problem, instead, the clauses of a variable often play different roles (in graph colouring, e.g., “at least one colour” for a node and “different colours” for the two ends of an edge), and one may expect a weighting of the neighbours to be of some use.

4 GAT-Q-SAT

GAT-Q-SAT inserts two graph attention layers [31] in the core block, between the node update and the global update of (3). Thus, the node embeddings are recomputed from those of their neighbours, with learned weights, before u' is computed. Each layer maps the embeddings h_i of the nodes into

$$h_i^+ = \bigg\| \sum_{k=1}^K \sum_{j \in \mathcal{N}_i} \alpha_{ij}^k W_k h_j + b, \quad \alpha_{ij}^k = \frac{\exp s_{ij}^k}{\sum_{l \in \mathcal{N}_i} \exp s_{il}^k}, \quad s_{ij}^k = \text{LeakyReLU}(\mathbf{a}_k^\top [W_k h_i \parallel W_k h_j \parallel W_k^e e'_{ij}]), \quad (4)$$

where $\mathcal{N}_i$ is the set of the neighbours of i in the bipartite graph (with no self-loop), $\parallel$ denotes concatenation, and e'_{ij} is the embedding of the edge between i and j computed by ϕ^e in the same application of the block. Hence, the coefficients α_{ij}^k depend on the current edge embeddings, which carry the polarity of the literal together with what the block has added to it. In our configuration the first layer has $K = 3$ heads of size 64 (concatenated into 192 components) and the second a single head, which returns to size 64; each layer is followed by a ReLU, and the output by a layer normalization. All the rest of the network (the encoder, the edge and node updates with their sum, the global update, the decoder, and M) is unchanged, and so is the

training configuration of section 2. In the terms of [2], the resulting block is a full graph-network block with a non-local block inserted between the node and the global update (cf. Figure 2).

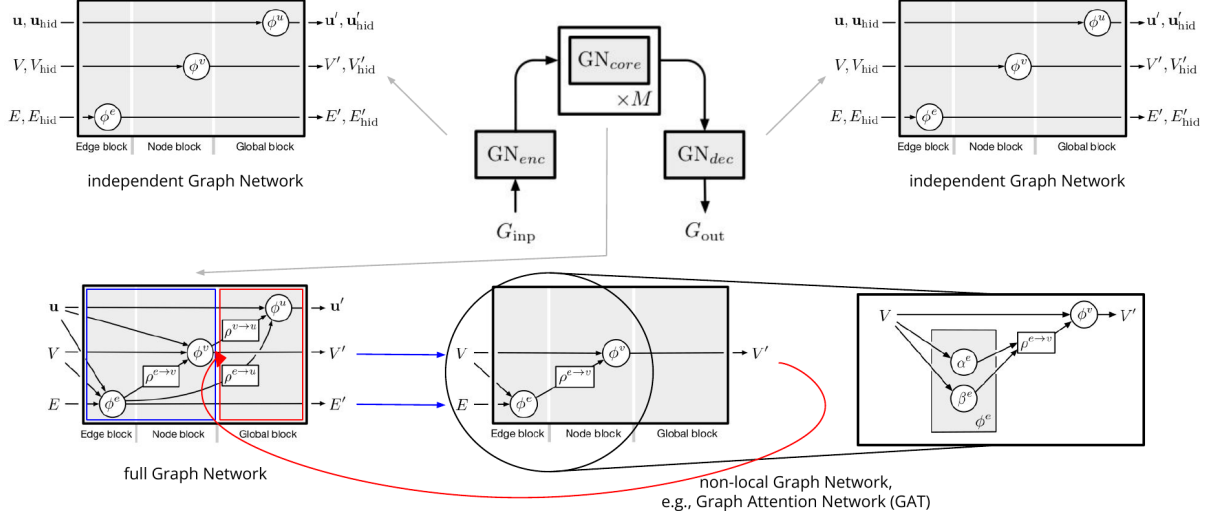


Figure 2: The core block of GAT-Q-SAT: the block of Figure 1 with a non-local block of two attention layers (4) inserted between the node and the global update. Blocks drawn after [2].

The attention layers add 42112 parameters to the 71144 of Graph-Q-SAT (about 60% more), and their cost is paid at each decision, since the coefficients are computed on all the edges at each application of the block. Of course, the coefficients can only be of use where the neighbours of a node differ in a way that matters for the decision. On a uniform random formula at the phase transition one may expect them to learn little (if anything), and the added parameters to make the fit somewhat harder. Note also that, since the layers are added and the sum of (3) is kept, section 6 compares the plain model with the same model plus a (larger) attention stage. Comparing two aggregation rules at the same capacity would require replacing the sum by the attention inside ϕ^v , which we have not done.

5 Implementation

The original implementation of [15] is written in PyTorch 1.1 with pinned versions of PyTorch Geometric [7] and of the compiled extensions `torch-scatter` and `torch-sparse`. It drives a modified MiniSat 2.2 [6], exposed to Python as a `gym` environment through a SWIG wrapper. These versions can no longer be installed together on a current system, and therefore we ported the code to current releases of PyTorch and PyTorch Geometric. In the port the aggregations of (3) are taken from `torch_geometric.utils.scatter`, which removes the two compiled extensions. The environment is constructed directly, without `gym.make` (whose checks reject the non-standard signatures of the MiniSat wrapper), which allows the same code to run with `gym` and with `gymnasium`; the SWIG wrapper is regenerated and rebuilt against `numpy 2`. Also, the version of PyTorch Geometric used in 2021 stores the linear map of an attention layer twice (`lin_src` and `lin_dst`), where the current one has a single `lin`. When a checkpoint of 2021 is loaded one of the two copies is kept; this loses nothing, since in all our checkpoints the two are equal.

In order to check that the port has not changed what the models compute, we re-ran on the new stack the evaluation of the checkpoints trained in 2021 on the original one, and compared the logs instance by instance. In 63 of the 86 repeated evaluations the number of iterations of the model coincides with the 2021 one on all the instances; in the other 23 the median ratio (5) changes by at most 0.03 (on `flat30-60` at cap 500, e.g., the medians are 1.833 for Graph-Q-SAT

and 2.000 for GAT-Q-SAT on both stacks). We have not traced the residual differences, which may be due to the non-determinism of the scatter operations on a GPU; they are in any case much smaller than the differences discussed in section 6. Indeed, inference exercises the graph construction, the message passing, the decoder and the stepping of the environment together, and hence this is a reasonably stringent test of all but the training path. The latter was checked by retraining the two colouring models from scratch on the new stack (these are the second seeds of section 6).

Finally, the evaluation code implements the three restricted heuristics of [29], which reduce the number of forward passes. In the first one the model takes the first N decisions and VSIDS the rest. In the second one a forward pass gives the K best actions, which are executed in order (skipping those whose variable has meanwhile been assigned) before the network is queried again. In the third one the Q -values computed at the root initialize the activities of VSIDS, which then runs unaided (this required a new method of the solver, which writes the activities and rebuilds the order heap). We do not report results for them, since on the instances of section 6 the search is too short for the saving in inference to show.

6 Computational results

We now compare the two models on test instances that are increasingly far from the training ones. As in [15], the quality of a heuristic is measured by the Median Relative Iteration Reduction

$$\text{MRIR} = \text{median} \frac{\# \text{iterations of MiniSat}}{\# \text{iterations of the model}}, \quad (5)$$

where the median is over the instances of a family, and an iteration is a branching decision (of the model or of VSIDS). Each evaluation is run with a “cap” on the number of decisions of the model, after which VSIDS takes the remaining ones; unless stated otherwise the cap is 500, as in training.

Reference solver. The model-guided solver runs without restarts, while MiniSat can be run with or without them (with the restart sequence of [19]), and the choice of the denominator of (5) is by no means inconsequential. On graph colouring MiniSat with restarts takes up to 26% fewer iterations than without (on `flat175-417`); on `uf250-1065` and `uuf250-1065`, however, it takes 2.2 and 2.9 times more. We divide by the stronger of the two on each family, i.e., by MiniSat with restarts on graph colouring (as in [15]) and without restarts on uniform random 3-SAT. The one exception is `uuf100-430`, where the run with restarts takes 6% fewer iterations but where our logs (those of 2021, the only ones for the random families) record only the other one; hence, the MRIR reported there is slightly favourable to the models. On the families of section 6.3 we also divide by the stronger of the two.

Runs. Each run is trained once on one family, with the configuration of section 2, and is evaluated at its last checkpoint. On graph colouring each variant has two runs trained on `flat50-115`, i.e., two seeds, one trained in 2021 on the original stack and one in 2026 on the ported one. On uniform random 3-SAT each variant has two runs trained on satisfiable formulas (one on `uf50-218`, one on `uf100-430`), whose results we average, and two trained on the unsatisfiable `uuf50-218` and `uuf100-430`, used only as a control in section 6.4. For a group of runs we report the mean of the MRIR of its runs.

Instances and machine. Each test set has 100 instances (101 for `flat150-360`): the held-out test part for the families used in training, and the whole family for the others (cf. the appendix). All the evaluations of 2026 were run on a Colab instance with a GPU, whose load we could not control. This does not affect the MRIR (a ratio of iteration counts), whereas the running times of section 6.5 are, and we discuss their reliability there.

6.1 Graph colouring

Table 1 reports the MRIR of the colouring-trained models on the eight graph-colouring families. Column “seeds” gives the MRIR of the run of 2021 and of the one of 2026, “mean” their mean, and “ Δ ” the difference of the means (positive when GAT-Q-SAT takes fewer decisions). Since the families range from 30 to 200 nodes, the largest test instances are four times the size of the training ones.

family	Graph-Q-SAT		GAT-Q-SAT		Δ
	seeds	mean	seeds	mean	
flat30-60	1.83 / –	1.83	2.00 / –	2.00	+0.17
flat50-115	2.18 / –	2.18	2.25 / –	2.25	+0.07
flat75-180	2.37 / –	2.37	2.94 / –	2.94	+0.58
flat100-239	2.64 / –	2.64	2.76 / –	2.76	+0.12
flat125-301	2.43 / –	2.43	3.09 / –	3.09	+0.66
flat150-360	1.74 / –	1.74	3.40 / –	3.40	+1.66
flat175-417	1.17 / –	1.17	2.59 / –	2.59	+1.42
flat200-479	1.05 / –	1.05	2.64 / –	2.64	+1.59

Table 1: Colouring-trained models on graph colouring: MRIR against MiniSat with restarts at cap 500, per seed (2021 / 2026) and mean over the seeds.

GAT-Q-SAT takes fewer decisions than Graph-Q-SAT on all eight families. Up to 100 nodes, which include the test part of the training family, the difference is between 0.07 and 0.58 and it does not grow with the size (it is 0.58 at 75 nodes and 0.12 at 100). From 125 nodes on it is between 0.66 and 1.66. This is mostly (but not only) due to the plain model, whose MRIR decreases from 2.64 at 100 nodes to 1.05 at 200 (i.e., almost to parity with the reference solver), while that of the attention model stays between 2.6 and 3.4. In other words, the two models are rather close on instances of about the training size, and the attention model degrades much less when the instances grow. With the second seed the pattern is the same on all the families, although the size of the difference changes from seed to seed by up to X .

Figure 3 shows the MRIR of the runs of 2021 as a function of the cap, family by family, on a common vertical scale (the runs of 2026 were evaluated only at the caps of Tables 1 and 3). On most families the plain model reaches its best MRIR at a cap of 50. It then degrades as it is allowed more decisions, down to 0.77 on flat200-479 at cap 1000; the MRIR of the attention model does not decrease with the cap on any family. On the larger instances, then, the decisions that the plain model takes after the first few tens make the search longer, and the ranking of Table 1 does not seem to depend on the choice of the cap.

6.2 A change of the training distribution

We now evaluate on graph colouring the models trained on satisfiable uniform random 3-SAT, whose formulas have none of the structure of a colouring problem. On five of the eight families the plain model is below the reference solver, its MRIR ranging from 0.63 to 1.16 (0.89 on average). The attention model, instead, is above it on seven (from 0.94 to 1.75, 1.44 on average); the difference between the two is 0.44 to 0.97 on the first seven families. On flat200-479, however, it drops to 0.10, and the attention model too falls slightly below parity. Figure 4 compares at cap 500 the three regimes that have been measured on the same families, and Figure 5 shows the models trained on random formulas as a function of the number of variables of the test instances.

Thus, the attention model is the better of the two at all sizes on graph colouring, and the worse one at all sizes on satisfiable random formulas (cf. section 6.4). The ranking may therefore

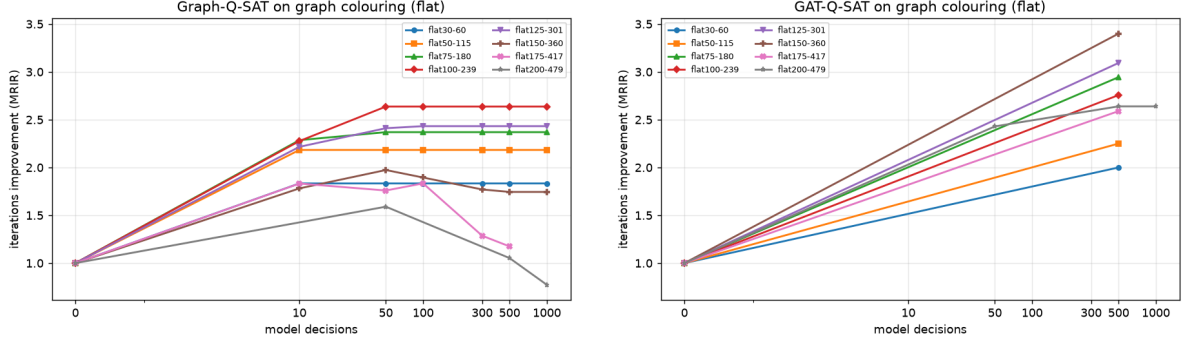


Figure 3: Colouring-trained runs of 2021: MRIR against MiniSat with restarts as a function of the cap, on the eight graph-colouring families, for Graph-Q-SAT (left) and GAT-Q-SAT (right), on a common vertical scale.

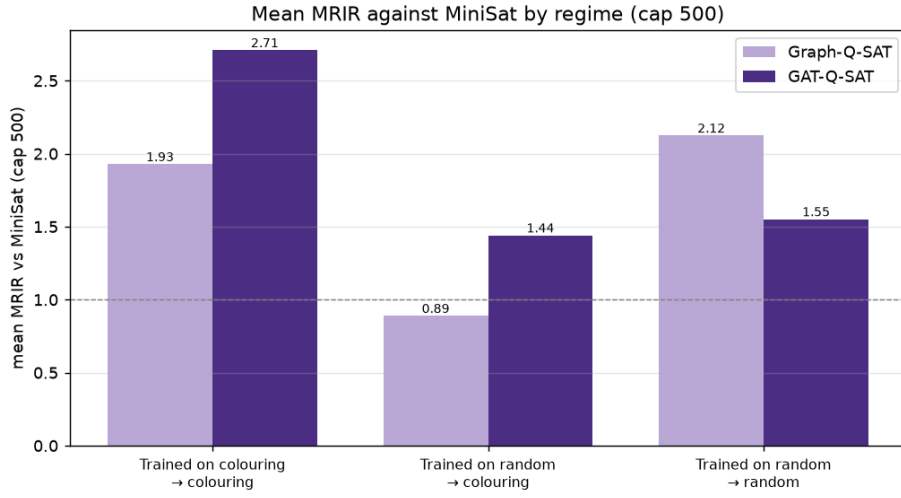


Figure 4: MRIR at cap 500, mean over the families: trained and tested on graph colouring (eight families), trained on satisfiable random 3-SAT and tested on graph colouring (eight families), and trained and tested on random 3-SAT (the six families of Table 2).

depend on the test family more than on the particular instances, and the learned weighting of the neighbours seems to be of some use even on a structure that the model has not seen in training. We remark, however, that the absolute difference is smaller here than in section 6.1 (0.55 against 0.78 on average), and that the MRIR of both models decreases with the size of the colouring instances.

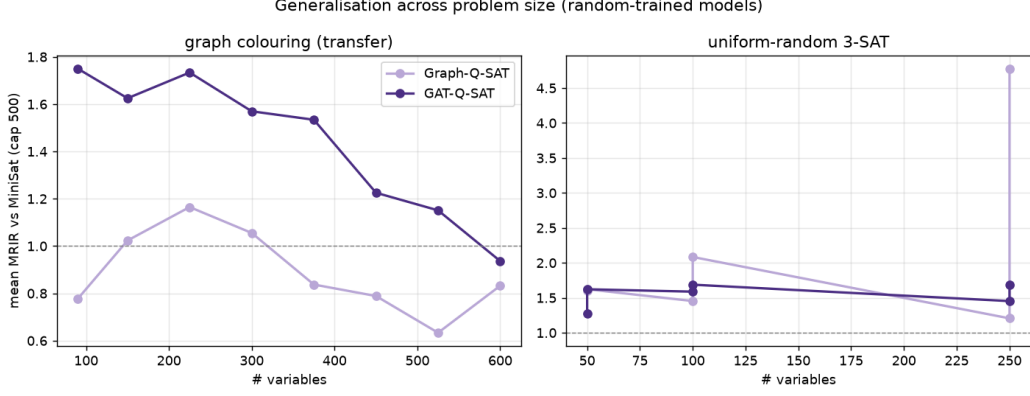


Figure 5: Models trained on satisfiable random 3-SAT: MRIR at cap 500 as a function of the number of variables of the test instances, on graph colouring (left) and on random 3-SAT (right, satisfiable and unsatisfiable families).

6.3 A change of domain

All the families above are random, either in the formula or in the graph. They say little about what a user of a learned heuristic would like to know, namely whether a heuristic trained on the instances at hand is of any use on different ones. We therefore evaluated the colouring-trained models of both seeds (with no retraining) on four SATLIB families of a different nature. These are small-world graph colouring [32] (a change of the topology of the graph within the same problem), the AIM instances [1] (structured 3-SAT with at most one solution), quasigroup completion [10], and planning [13] (blocks world and logistics). These families are “small” (60, 72, 11 and 8 instances respectively), and we evaluate them at a cap of 200 to contain the cost of the largest planning and quasigroup instances. Figure 6 reports the MRIR, divided by the stronger MiniSat run on each family (with restarts on small-world colouring and on quasigroups, without on planning, the two being equal on AIM).

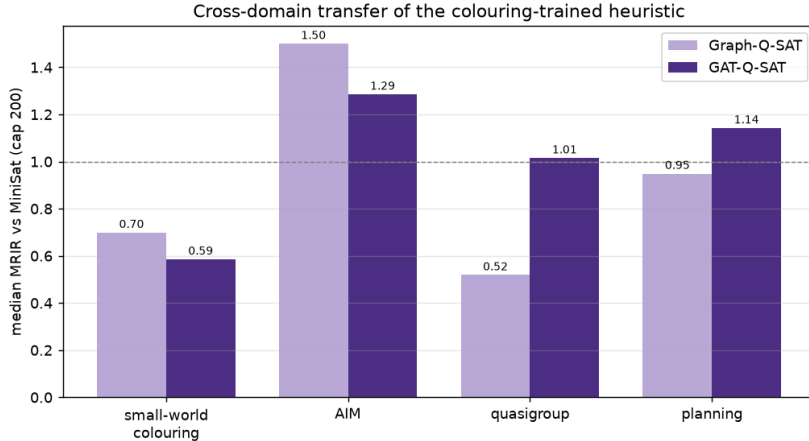


Figure 6: Colouring-trained models on four families not seen in training: MRIR at cap 200 against the stronger MiniSat run on each family, mean over the two seeds.

On AIM (1.49), quasigroups (1.08) and planning (1.14) the attention model is above the reference solver, and the plain one only on AIM (1.50). On small-world colouring, which is the closest of the four to the training problem, both are below it (0.58 and 0.81). On AIM the two models are basically equivalent, while the advantage of the attention model on quasigroups and planning rests on 11 and 8 instances, and it should therefore be taken as an indication

only. Our result on small-world colouring is somewhat unexpected, since a change of topology within the same problem hurts both models more than a change of problem does. We do not have an explanation for it. One may speculate that the models might be sensitive to the degree distribution of the graph, which is very different in the (almost regular) small-world graphs and in the random graphs of the `flat` families.

6.4 Uniform random 3-SAT

On uniform random 3-SAT the ranking of the two models is reversed. Table 2 reports the MRIR of the models trained on satisfiable formulas (mean of the runs trained on `uf50-218` and `uf100-430`) on the satisfiable and on the unsatisfiable families of the same generator. Figure 7 shows the same models as a function of the cap. On the satisfiable families the plain model is better at all sizes: by 0.35 at 50 variables, by 0.49 at 100, and by 3.09 at 250, where its MRIR is 4.78 against 1.69. (The plain model trained on `uf50-218` alone reaches 6.19 there, on instances five times larger than the training ones.) This is consistent with the remark of section 4: at the phase transition the neighbours of a node are statistically alike, and the attention coefficients have little to learn.

	satisfiable			unsatisfiable		
	uf50-218	uf100-430	uf250-1065	uuf50-218	uuf100-430	uuf250-1065
Graph-Q-SAT	1.62	2.08	4.78	1.61	1.45	1.21
GAT-Q-SAT	1.27	1.59	1.69	1.62	1.69	1.45

Table 2: Models trained on satisfiable random 3-SAT (mean of the runs trained on `uf50-218` and `uf100-430`): MRIR against MiniSat without restarts at cap 500.

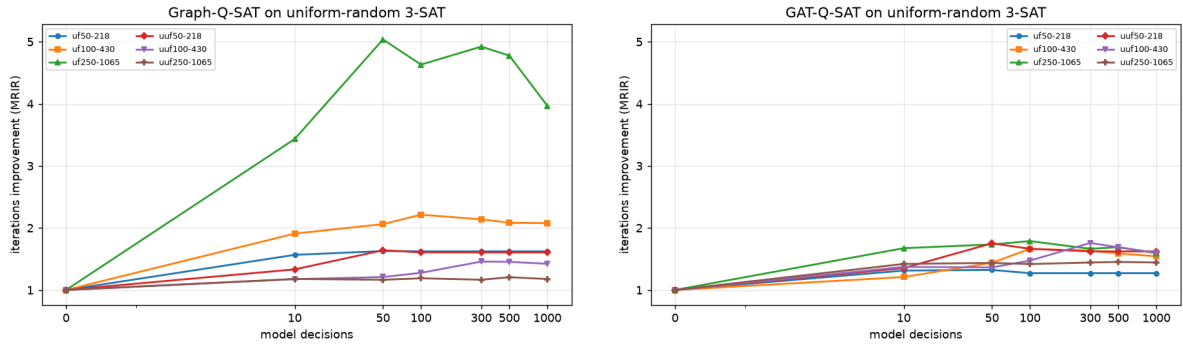


Figure 7: Models trained on satisfiable random 3-SAT (mean of the two runs): MRIR against MiniSat without restarts as a function of the cap, on the satisfiable (`uf`) and unsatisfiable (`uuf`) families, for Graph-Q-SAT (left) and GAT-Q-SAT (right), on a common vertical scale.

However, the advantage of the plain model does not carry over to the unsatisfiable formulas. On these the two models are level at 50 variables (1.61 and 1.62), and the attention model is ahead at 100 (1.69 against 1.45) and at 250 (1.45 against 1.21). Two explanations are possible, and our measurements do not separate them: either the plain model may have learned something about where the satisfying assignments of these formulas tend to be, which is of no use when there are none, or proving unsatisfiability leaves little room to the branching heuristic, so that the two models end up closer. In fact, the control runs trained on unsatisfiable formulas improve the plain model (from 1.61 to 1.96 on `uuf50-218` and from 1.45 to 1.56 on `uuf100-430`) but not the attention one (1.63 and 1.50). All in all, the regime in which the plain aggregation is

clearly superior is that of satisfiable uniform random formulas, which is also, typically, the one with the least structure for a weighting of the neighbours to exploit.

6.5 Iterations and running time

The MRIR counts iterations, which are the quantity on which the heuristic acts. However, a decision of the model costs a forward pass of the network on the current graph, and the attention layers make that pass more expensive. We measure the time per instance around the episode. It includes the parsing of the formula, the variable elimination, the construction of the graph at each decision, the forward passes and the part of the search left to VSIDS (but not the loading of the model and the runs of MiniSat that give the denominator of (5), which are computed once per family and stored with it). We report medians over the instances, since a few instances usually take much longer than the others. As said above, the machine was not under our control: repeating two of the evaluations gave medians within $X\%$ of the first ones, and hence we only comment on differences much larger than that.

	cap 50		cap 500		cap 1000	
	sec	MRIR	sec	MRIR	sec	MRIR
Graph-Q-SAT	2.55	1.59	26.51	1.05	57.67	0.77
GAT-Q-SAT	3.52	2.43	12.20	2.64	12.19	2.64

Table 3: Colouring-trained models on **flat200-479**: median seconds per instance (“sec”) and MRIR against MiniSat with restarts, as a function of the cap (mean over the seeds).

Table 3 reports time and MRIR on the largest colouring family at three values of the cap. At cap 50 a decision of GAT-Q-SAT is more expensive than one of Graph-Q-SAT (3.52 against 2.55 seconds for the same number of model decisions), as expected. At larger caps, instead, the attention model is the faster of the two (by a factor of 2 at cap 500 and of almost 5 at cap 1000), since it terminates in fewer decisions; indeed, each decision of a model costs far more than the VSIDS decisions it replaces. The plain model pays 22 times the time of cap 50 to reach a lower MRIR, and it ends up below the reference solver at cap 1000. Going from cap 50 to cap 500 improves the attention model by 0.21, and going to 1000 changes nothing, since its episodes terminate before the cap is reached.

In absolute terms both models take seconds per instance on formulas with at most a few thousand clauses, which MiniSat solves in a small fraction of a second. At this scale neither model is competitive with the solver in running time, and the reduction of the iterations measures the quality of the decisions (their convenience being a different matter). This is the motivation of the restricted heuristics of [29] (cf. section 5) and of the offline inference of [34], whose benefit may only show on instances where the search is long enough to dominate the cost of inference.

7 Conclusions

We have added two edge-featured graph attention layers to the core block of Graph-Q-SAT, and compared the resulting model with the plain one at matched training set and configuration. On all the graph-colouring families the attention model took fewer decisions. The difference was small on instances of about the training size and large (up to about 1.7) on those three and four times larger, where the plain model approached the reference solver. It was also the better of the two when trained on random formulas and tested on graph colouring and, with the caveats due to the size of the families, on some of the structured families that neither model had seen. On satisfiable uniform random 3-SAT, instead, the plain model was clearly superior, consistently with the absence of structure for the attention to weight. On the unsatisfiable

formulas of the same generator, however, the two were level or the attention model was slightly ahead. Within the limits of these experiments, the attention layers seem to make the learned heuristic less sensitive to the size and to the nature of the test instances when these have some structure, at the cost of about 60% more parameters and of a more expensive decision.

Several limitations should be kept in mind. First, the comparison involves two training seeds per variant on graph colouring and one or two on the other families. The transfer study rests on four “small” families, two of which have 11 and 8 instances; hence, differences of a few tenths of MRIR may well be within the variance of DQN training. Furthermore, since the attention layers are added to the block, the comparison does not separate the effect of the weighting of the neighbours from that of the added capacity, which may well account for part of the difference; replacing the sum of (3) by the attention, at a comparable number of parameters, would settle this point. On the instances used here neither model is competitive with MiniSat in running time. Also, the restricted heuristics that should reduce the cost of inference are implemented but have not been measured on instances large enough for them to matter. Finally, the poor transfer of both models to small-world colouring (the family closest to the training one) is not explained. It is perhaps the first thing that a larger panel of instances, e.g., that of [17], should clarify; we hope to address some of these points in the future.

Acknowledgements

The implementation that we started from is the one released by Kurin et al. with [15], and the solver underneath it is MiniSat [6].

References

- [1] Yuichi Asahiro, Kazuo Iwama, and Eiji Miyano. Random generation of test instances with controlled attributes. In *Cliques, Coloring, and Satisfiability*, volume 26 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, pages 377–393. American Mathematical Society, 1996.
- [2] Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, et al. Relational inductive biases, deep learning, and graph networks. *arXiv:1806.01261*, 2018.
- [3] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: A methodological tour d’horizon. *European Journal of Operational Research*, 290(2):405–421, 2021.
- [4] Shaked Brody, Uri Alon, and Eran Yahav. How attentive are graph attention networks? In *International Conference on Learning Representations*, 2022.
- [5] Joseph C. Culberson and Feng Luo. Exploring the k -colorable landscape with iterated greedy. In *Cliques, Coloring, and Satisfiability*, volume 26 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, pages 245–284. American Mathematical Society, 1996.
- [6] Niklas Eén and Niklas Sörensson. An extensible SAT-solver. In *Theory and Applications of Satisfiability Testing (SAT)*, pages 502–518, 2003.
- [7] Matthias Fey and Jan E. Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

- [8] Maxime Gasse, Didier Chételat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems 32*, pages 15554–15566, 2019.
- [9] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning*, 2017.
- [10] Carla P. Gomes and Bart Selman. Problem structure in the presence of perturbations. In *Proceedings of the Fourteenth National Conference on Artificial Intelligence (AAAI)*, pages 221–226, 1997.
- [11] Wenxuan Guo, Hui-Ling Zhen, Xijun Li, Wanqian Luo, Mingxuan Yuan, Yaohui Jin, and Junchi Yan. Machine learning methods in solving the Boolean satisfiability problem. *Machine Intelligence Research*, 20(5):640–655, 2023.
- [12] Holger H. Hoos and Thomas Stützle. SATLIB: An online resource for research on SAT. In *SAT 2000*, pages 283–292. IOS Press, 2000.
- [13] Henry Kautz and Bart Selman. Pushing the envelope: Planning, propositional logic, and stochastic search. In *Proceedings of the Thirteenth National Conference on Artificial Intelligence (AAAI)*, pages 1194–1201, 1996.
- [14] Elias B. Khalil, Pierre Le Bodic, Le Song, George L. Nemhauser, and Bistra Dilkina. Learning to branch in mixed integer programming. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, 2016.
- [15] Vitaly Kurin, Saad Godil, Shimon Whiteson, and Bryan Catanzaro. Can Q-learning with graph networks learn a generalizable branching heuristic for a SAT solver? In *Advances in Neural Information Processing Systems 33*, 2020.
- [16] Michail G. Lagoudakis and Michael L. Littman. Learning to select branching rules in the DPLL procedure for satisfiability. *Electronic Notes in Discrete Mathematics*, 9:344–359, 2001.
- [17] Zhaoyu Li, Jinpei Guo, and Xujie Si. G4SATBench: Benchmarking and advancing SAT solving with graph neural networks. In *Advances in Neural Information Processing Systems 36*, 2023.
- [18] Jia Hui Liang, Vijay Ganesh, Pascal Poupart, and Krzysztof Czarnecki. Learning rate based branching heuristic for SAT solvers. In *Theory and Applications of Satisfiability Testing (SAT)*, pages 123–140, 2016.
- [19] Michael Luby, Alistair Sinclair, and David Zuckerman. Optimal speedup of Las Vegas algorithms. *Information Processing Letters*, 47(4):173–180, 1993.
- [20] João Marques-Silva, Inês Lynce, and Sharad Malik. Conflict-driven clause learning SAT solvers. In *Handbook of Satisfiability*, Frontiers in Artificial Intelligence and Applications, chapter 4. IOS Press, second edition, 2021.
- [21] João P. Marques-Silva and Karem A. Sakallah. GRASP: A search algorithm for propositional satisfiability. *IEEE Transactions on Computers*, 48(5):506–521, 1999.
- [22] David G. Mitchell, Bart Selman, and Hector J. Levesque. Hard and easy distributions of SAT problems. In *Proceedings of the Tenth National Conference on Artificial Intelligence (AAAI)*, pages 459–465, 1992.

- [23] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.
- [24] David Mojžišek, Jan Hula, Ziwei Li, Chunlin Zhou, and Mikoláš Janota. Neural approaches to SAT solving: Design choices and interpretability. *arXiv:2504.01173*, 2025.
- [25] Matthew W. Moskewicz, Conor F. Madigan, Ying Zhao, Lintao Zhang, and Sharad Malik. Chaff: Engineering an efficient SAT solver. In *Proceedings of the 38th Design Automation Conference (DAC)*, pages 530–535, 2001.
- [26] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations*, 2016.
- [27] Daniel Selsam and Nikolaj Bjørner. Guiding high-performance SAT solvers with unsat-core predictions. In *Theory and Applications of Satisfiability Testing (SAT)*, 2019.
- [28] Daniel Selsam, Matthew Lamm, Benedikt Bünz, Percy Liang, Leonardo de Moura, and David L. Dill. Learning a SAT solver from single-bit supervision. In *International Conference on Learning Representations*, 2019.
- [29] Mikhail Shirokikh, Ilya Shenbin, Anton Alekseev, and Sergey Nikolenko. Machine learning for SAT: Restricted heuristics and new graph representations. *arXiv:2307.09141*, 2023.
- [30] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, 2016.
- [31] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.
- [32] Toby Walsh. Search in a small world. In *Proceedings of the Sixteenth International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1172–1177, 1999.
- [33] Fei Wang and Tiark Rompf. From gameplay to symbolic reasoning: Learning SAT solver heuristics in the style of Alpha(Go) Zero. *arXiv:1802.05340*, 2018.
- [34] Wenxi Wang, Yang Hu, Mohit Tiwari, Sarfraz Khurshid, Kenneth L. McMillan, and Risto Miikkulainen. NeuroBack: Improving CDCL SAT solving using graph neural networks. In *International Conference on Learning Representations*, 2024.
- [35] Emre Yolcu and Barnabás Póczos. Learning local search heuristics for Boolean satisfiability. In *Advances in Neural Information Processing Systems 32*, pages 7990–8001, 2019.
- [36] Wenjie Zhang, Zeyu Sun, Qihao Zhu, Ge Li, Shaowei Cai, Yingfei Xiong, and Lu Zhang. NLocalSAT: Boosting local search with solution prediction. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1177–1183, 2020.

A The instances, and how the numbers are regenerated

All the instances come from SATLIB [12], in DIMACS form, and are stored in the repository of the note. The families used in training (flat50-115, uf50-218, uf100-430, uuf50-218 and uuf100-430) are split once, with a fixed seed, into training, validation and test parts of 80%, 10% and 10% (800, 100 and 100 formulas for most of them). The evaluations of

section 6 use the test parts of these families and the whole of the others, i.e., `flat30-60`, `flat75-180`, `flat100-239`, `flat125-301`, `flat150-360`, `flat175-417` and `flat200-479` for graph colouring, and `uf250-1065` and `uuf250-1065` for random 3-SAT, with 100 formulas each (101 for `flat150-360`).

The four families of section 6.3 are the following: (i) small-world colouring, 60 instances with 500 variables and 3100 clauses, obtained by 5-colouring graphs on 100 vertices whose rewiring parameter moves them from a ring lattice towards a random graph (the six least random levels of SATLIB, 10 instances each); (ii) AIM, 72 instances with 50, 100 and 200 variables, 48 satisfiable with a single solution and 24 unsatisfiable; (iii) quasigroup completion, 11 instances with 343 to 729 variables and up to about $1.5 \cdot 10^5$ clauses; (iv) planning, 8 instances with 48 to 1141 variables from the blocks-world and logistics encodings. None of them is used in training.

For each family the iterations of MiniSat with and without restarts are computed once, instance by instance, and stored with the family, so that the denominator of (5) is the same for all the models. The evaluation of a model on a family at a cap writes one line per instance, with the time, the iterations of the model and the two iterations of MiniSat. The logs of 2021, which are the only ones for the random families, record only the ratio against MiniSat without restarts. The tables of section 6 are printed by `paper_numbers.py`, which reads the logs, groups the runs by variant and training family, and applies the choice of the denominator described at the beginning of section 6. The figures are drawn from the same logs, and with the same grouping, by `make_plots.py` (Figures 3 and 7), `paper_analysis.py` (Figures 4 and 5) and `transfer_study.py` (Figure 6), which also runs the evaluations of section 6.3; those of the other sections are run by `reeval.sh`. The logs written in 2026 are kept with the checkpoints, outside the repository.